

Lab 2: Text Pre-processing and Regular Expressions

Student Name: Faisal AL Zahrani

Student ID: 2230000363

```
In [1]: # =====  
# Import Required Libraries  
# =====  
  
import re  
import pandas as pd  
import nltk  
import spacy  
  
from collections import Counter  
  
nltk.download("punkt", quiet=True)  
nltk.download("punkt_tab", quiet=True)  
  
print("Libraries imported successfully.")
```

Libraries imported successfully.

Task 1: Hashtag Extraction and Analysis

In this task, the Apple Twitter dataset is analyzed using regular expressions.

The objectives are to:

- Extract all hashtags from the tweets.
- Calculate the total number of hashtags.
- Calculate the number of unique hashtags.
- Identify the top 10 most frequently used hashtags.

The regular expression `#\w+` is used to detect hashtags.

- `#` matches the hashtag symbol.
- `\w` matches letters, digits, and underscores.
- `+` means one or more characters.

The tweets are converted to lowercase so that hashtags such as `#Apple` and `#apple` are treated as the same hashtag.

The `Counter` class is then used to calculate the frequency of each hashtag and display the top 10 most frequently used hashtags.

```
In [3]: # =====
# Task 1: Load Apple Twitter Dataset
# =====

DATASET_PATH = "apple-twitter-sentiment-texts.csv"

df = pd.read_csv(DATASET_PATH)

print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())

display(df.head())
```

Dataset loaded successfully.

Dataset shape: (1630, 2)

Columns: ['text', 'sentiment']

	text	sentiment
0	Wow. Yall needa step it up @Apple RT @heynyla:...	-1
1	What Happened To Apple Inc? http://t.co/FJEX...	0
2	Thank u @apple I can now compile all of the pi...	1
3	The oddly uplifting story of the Apple co-foun...	0
4	@apple can i exchange my iphone for a differen...	0

```

In [4]: # =====
# Task 1: Identify Tweet Text Column
# =====

possible_columns = [
    "text",
    "tweet",
    "tweets",
    "tweet_text",
    "sentiment_text"
]

text_column = None

for column in possible_columns:
    if column in df.columns:
        text_column = column
        break

if text_column is None:
    raise ValueError(
        f"Tweet text column not found. Available columns: {df.columns.tolist()}"
    )

print("Tweet text column:", text_column)

```

Tweet text column: text

```

In [5]: # =====
# Task 1: Extract Hashtags Using Regular Expressions
# =====

hashtag_pattern = r"#\w+"

all_hashtags = []

for tweet in df[text_column].dropna().astype(str):

    # Convert text to Lowercase to avoid counting
    # #Apple and #apple as different hashtags

```

```
tweet = tweet.lower()

hashtags = re.findall(hashtag_pattern, tweet)

all_hashtags.extend(hashtags)

print("Total number of hashtags:", len(all_hashtags))
print("Number of unique hashtags:", len(set(all_hashtags)))
```

Total number of hashtags: 1616

Number of unique hashtags: 555

```
In [6]: # =====
# Task 1: Top 10 Most Frequently Used Hashtags
# =====

hashtag_counts = Counter(all_hashtags)

top_10_hashtags = hashtag_counts.most_common(10)

top_10_df = pd.DataFrame(
    top_10_hashtags,
    columns=["Hashtag", "Frequency"]
)

print("Top 10 Most Frequently Used Hashtags:")
display(top_10_df)
```

Top 10 Most Frequently Used Hashtags:

	Hashtag	Frequency
0	#aapl	400
1	#apple	162
2	#iphone	38
3	#december	37
4	#iphone6	29
5	#stocks	15
6	#ipad	14
7	#iphone6plus	13
8	#iphone6s	13
9	#tech	12

Task 2: Using `re.compile()`

In this task, `re.compile()` is used to create a reusable regular expression pattern.

The pattern `\d+` matches one or more digits.

The compiled pattern is used to replace the year `2021` with `2022`.

`re.compile()` is useful because it converts a regular expression into a reusable pattern object. This makes the code cleaner and more efficient when the same pattern is used multiple times.

```
In [8]: # =====  
# Task 2: Using re.compile()  
# =====  
  
text = "This year is 2021"
```

```
# Create a compiled regex pattern that matches one or more digits
pattern = re.compile(r"\d+")

# Print the type of the compiled pattern
print("Pattern type:", type(pattern))

# Replace 2021 with 2022
updated_text = pattern.sub("2022", text)

print("Original text:", text)
print("Updated text :", updated_text)
```

Pattern type: <class 're.Pattern'>

Original text: This year is 2021

Updated text : This year is 2022

Task 3: Using `re.split()`

In this task, `re.split()` is used to split a string based on a regular expression pattern.

The pattern `\d+` matches one or more digits.

Therefore, the numbers `11`, `2`, `3`, and `4` are used as splitting points in the text.

`re.split()` is useful when we want to divide text using a pattern instead of a single fixed character.

```
In [10]: # =====
# Task 3: Using re.split()
# =====

text = "a 11 b 2 3 c 4"

# Split the text whenever one or more digits appear
result = re.split(r"\d+", text)
```

```
print("Original text:", text)
print("Split result :", result)
```

Original text: a 11 b 2 3 c 4

Split result : ['a ', ' b ', ' ', ' c ', '']

Task 4: Tokenization Using spaCy and NLTK

In this task, the sentence "I'm enjoying the NLP course!" is tokenized using both spaCy and NLTK.

The function `spacy.load("en_core_web_sm")` loads the small English language model provided by spaCy.

This model provides several NLP features, including:

- Tokenization
- Part-of-speech tagging
- Lemmatization
- Named Entity Recognition

The same sentence is also tokenized using `nltk.word_tokenize()`.

For this sentence, spaCy and NLTK produce the same tokens. However, both libraries use different tokenization rules, so their outputs may differ when processing more complex text.

```
In [12]: # =====
# Task 4: Tokenization Using spaCy
# =====

# Load the small English model
nlp = spacy.load("en_core_web_sm")

text = "I'm enjoying the NLP course!"

# Process the sentence
doc = nlp(text)

print("spaCy Tokens:")
```

```
print("-" * 20)

for token in doc:
    print(token.text)
```

spaCy Tokens:

```
-----
I
'm
enjoying
the
NLP
course
!
```

```
In [13]: # =====
# Task 4: Tokenization Using NLTK
# =====

text = "I'm enjoying the NLP course!"

nltk_tokens = nltk.word_tokenize(text)

print("NLTK Tokens:")
print("-" * 20)

for token in nltk_tokens:
    print(token)
```

NLTK Tokens:

```
-----
I
'm
enjoying
the
NLP
course
!
```

```
In [15]: # =====
# Task 4: Compare spaCy and NLTK Tokenization
```

```
# =====  
  
spacy_tokens = [token.text for token in nlp(text)]  
nltk_tokens = nltk.word_tokenize(text)  
  
print("spaCy Tokens:")  
print(spacy_tokens)  
  
print("\nNLTK Tokens:")  
print(nltk_tokens)  
  
print("\nNumber of spaCy tokens:", len(spacy_tokens))  
print("Number of NLTK tokens :", len(nltk_tokens))
```

spaCy Tokens:

['I', "'m", 'enjoying', 'the', 'NLP', 'course', '!']

NLTK Tokens:

['I', "'m", 'enjoying', 'the', 'NLP', 'course', '!']

Number of spaCy tokens: 7

Number of NLTK tokens : 7